

1 Config Generator

The generator creates ROS2-Monitor-Infra runtime YAML from explicit user choices. The user does not manually write YAML files. Instead, the user interface collects the choices, serialises them into a single intermediate representation (a JSON *deployment plan*), and the generator turns that plan into one or more runtime YAML files.

The deployment plan is the contract between the front end and the generator. When the user finishes selecting in the interface, the resulting network request carries exactly this JSON document; the generator consumes it as its sole input. The generator never inspects a deployment-mode name and never applies a hidden, mode-specific template. It performs a deterministic projection of the plan onto YAML, so the same code path produces every supported architecture.

1. discover visible ROS2 topics, services, and actions;
2. let the user select the resources to monitor;
3. let the user select custom converter and verdict classes;
4. let the user place observations and monitors onto locations and connect those locations with transport links;
5. serialise all of the above into the JSON deployment plan;
6. project the plan: emit one runtime YAML file per location.

2 Why a Deployment Plan Instead of a Deployment Mode

A single “deployment mode” selector is not expressive enough to describe the range of monitoring architectures this framework targets, and it hides the decisions that actually determine the generated files. Following the organisations identified for distributed runtime verification by Francalanza, Pérez, and Sánchez (Section 4 of *Runtime Verification for Decentralised and Distributed Systems*), an architecture is determined by three independent choices, not by one enumerated name:

- **Placement.** At which *location* is each observation collected, and at which location is each monitor evaluated? A monitor may sit with its observations or be pushed to a remote location.
- **What crosses a location boundary.** A link between two locations may carry raw trace events (*DataRecords*) or already-computed *verdicts*. This single choice is what distinguishes an orchestrated architecture from a choreographed one.
- **Transport.** How a link is realised: over a message bus (MQTT) for online monitoring, or through a file for offline replay.

The deployment plan makes these three choices explicit and first-class. A named mode (“integrated local”, “central over MQTT”, “hybrid”, “choreographed verdict aggregation”, “offline replay”) is therefore not a special case in the generator; it is merely a *preset* that pre-fills the plan. Reference architectures outside the named presets are expressed by writing the plan directly, with no change to the generator.

3 Intermediate Representation

The deployment plan separates *what is monitored* (transport-independent) from *where and how it runs* (the topology). The first two sections below form a catalogue that is reusable across architectures; the last two sections place that catalogue onto a concrete distributed topology.

Plan section	Role
observations	Transport-independent catalogue of ROS2 sources to observe (topics, services, actions) with their optional per-source transformers. Each has a stable id . Corresponds to the sources of a local <i>trace</i> .
monitors	Transport-independent catalogue of monitoring chains. Each is one converter $\rightarrow$ verdict pair with its class references and constructor parameters, plus the observation (or upstream verdict) ids it consumes . Each has a stable id .
locations	The placement nodes. Each location becomes exactly one generated runtime YAML file (one runtime process). It declares which observations it captures live, which monitors it hosts , and its output settings.
links	The typed edges between locations. Each link declares a payload (records or verdicts), a transport (mqtt or file), and which catalogue items it carries .

3.1 Plan schema

The abstract shape of the plan is shown below. Angle brackets denote values filled from user choices; they are not literal.

```
{
  "schema_version": "1.0",

  "observations": [
    {
      "id":          "<stable observation id>",
      "kind":        "topic | service | action",
      "name":        "<ROS graph name>",
      "type":        "<ROS2 message/service/action type>",
      "transformers": [ { "type": "<built-in transformer>", "kwarg": "<value>" } ],
      "qos":         "<topic only, optional>",
      "phases":      "<action only, optional>"
    }
  ],

  "monitors": [
    {
      "id":          "<stable monitor id>",
      "converter": { "type": "<module.path:ClassName>", "params": { } },
      "verdict":   { "type": "<module.path:ClassName>", "params": { } },
      "consumes":  [ "<observation id | monitor id>" ]
    }
  ],

  "locations": [
    {
```

```

    "id":      "<stable location id>",
    "output":  { "output_dir": "<path>", "session_id_prefix": "<prefix>" },
    "captures": [ "<observation id>" ],
    "hosts":   [ "<monitor id>" ]
  }
],

"links": [
  {
    "id":      "<stable link id>",
    "from":     "<location id>",
    "to":       "<location id>",
    "payload":  "records | verdicts",
    "transport": { "type": "mqtt | file", "<setting>": "<value>" },
    "carries":  [ "<observation id | monitor id>" ]
  }
]
}

```

A location that **captures** live observations generates a **monitor_node** configuration; a location that only receives a **records** link generates a **verdict_runner** configuration. The generator derives this from the plan, never from a mode flag.

4 Architectures as Placement Patterns

Each of the four monitoring organisations is a constraint on the plan, not a distinct generator path. The table states each organisation purely in terms of where monitors are placed and what the links carry.

Organisation	Locations	Links carry	Monitor placement
Traditional / centralised (§4.1)	one	none	monitors co-located with their observations.
Decentralised (§4.2)	many, shared clock	records (or a shared trace)	monitors partitioned across locations over one logical trace.
Orchestrated (§4.3)	N capture, 1 analysis	records to the analysis location	all monitors hosted at the central analysis location.
Choreographed (§4.4)	N capture+analyse (+ aggregator)	verdicts to peers / aggregator	local monitors at each capture location; aggregation monitors at the aggregator.

The named presets map onto these organisations as follows: integrated local is §4.1; central over MQTT is §4.3 with an **mqtt** record link; offline replay is §4.3 with a **file** record link; choreographed verdict aggregation is §4.4; the hybrid preset is a combination that both evaluates local monitors (§4.4 verdict export) and exports raw records for central re-evaluation (§4.3 record export) from the same capture location.

5 Config Generation Algorithm

The generator projects the plan onto YAML, one file per location. The projection is total and architecture-agnostic: links, not a mode name, decide which ingress source and which egress exporters appear in each file.

Algorithm 1 Generate runtime YAML from a deployment plan

Require: deployment plan Π (**observations**, **monitors**, **locations**, **links**)

Ensure: one runtime YAML document per location

```

1:  $F \leftarrow \emptyset$  ▷ map from file name to YAML document
2: for all location  $L$  in  $\Pi.locations$  do
3:    $Y \leftarrow \text{NEWYAML}$ 
4:    $Y.monitor \leftarrow L.output$ 
5:   for all observation id  $o$  in  $L.captures$  do
6:      $obs \leftarrow \Pi.observations[o]$ 
7:      $entry \leftarrow \text{MAKESOURCEENTRY}(obs.name, obs.kind, obs.type)$ 
8:      $\text{APPLYOPTIONALSETTINGS}(entry, obs)$  ▷ transformers, qos, phases
9:     add  $entry$  to  $Y[obs.kind]$  ▷ topics / services / actions
10:  end for
11:   $in \leftarrow \text{INGRESSRECORDLINK}(\Pi, L)$ 
12:  if  $in \neq \text{none}$  then
13:     $Y.verdict\_runner.source \leftarrow \text{TRANSPORTSOURCE}(in.transport)$ 
14:  end if
15:  for all monitor id  $m$  in  $L.hosts$  do
16:     $mon \leftarrow \Pi.monitors[m]$ 
17:     $chain \leftarrow \text{MAKECONVERTERENTRY}(mon.converter)$ 
18:     $chain.inputs \leftarrow \text{RESOLVEINPUTNAMES}(mon.consumes, \Pi, L)$ 
19:     $chain.verdict \leftarrow \text{MAKEVERDICTBLOCK}(mon.verdict)$ 
20:     $chain.verdict.exporters \leftarrow \text{VERDICTEXPORTERS}(\Pi, L, mon)$ 
21:    add  $chain$  to  $Y.converters$ 
22:  end for
23:  for all link  $e$  in  $\text{EGRESSRECORDLINKS}(\Pi, L)$  do
24:     $\text{ATTACHRECORDEXPORTER}(Y, e, \Pi)$ 
25:  end for
26:   $F[\text{FILENAME}(L)] \leftarrow Y$ 
27: end for
28:  $\text{WRITEYAMLFILES}(F)$ 
29: return  $F$ 

```

6 Runtime Resource Scanning

Discovery feeds the **observations** catalogue during selection. It is independent of the topology.

Algorithm 2 Scan ROS2 resources

Ensure: resource catalogue G

```
1:  $T \leftarrow \text{run ros2 topic list -t}$ 
2:  $V \leftarrow \text{run ros2 service list -t}$ 
3:  $A \leftarrow \text{run ros2 action list -t}$ 
4:  $G \leftarrow \emptyset$ 
5: for all topic  $(name, type)$  in  $T$  do
6:   add  $(topic, name, type)$  to  $G$ 
7: end for
8: for all service  $(name, type)$  in  $V$  do
9:   add  $(service, name, type)$  to  $G$ 
10: end for
11: for all action  $(name, type)$  in  $A$  do
12:   add  $(action, name, type)$  to  $G$ 
13: end for
14: return sorted  $G$ 
```

7 Projection Rules

Each YAML file is assembled from four kinds of plan element. The rules below define the mapping precisely; the named presets differ only in which rules fire, because they differ only in their links.

7.1 Location output

`location.output` becomes the `monitor` block. `output_dir` and `session_id_prefix` are copied verbatim.

7.2 Captured observations

Each `id` in `location.captures` is resolved against `observations` and copied into the matching section by its kind:

- a `topic` observation becomes an entry under `topics`;
- a `service` observation becomes an entry under `services`;
- an `action` observation becomes an entry under `actions`.

```
monitor:
  output_dir: <location.output.output_dir>
  session_id_prefix: <location.output.session_id_prefix>
```

```
topics:
- name: <observation.name>
  type: <observation.type>
  qos: <observation.qos, optional>
```

```
actions:
- name: <observation.name>
  type: <observation.type>
  phases: <observation.phases, optional>
```

7.3 Hosted monitors

Each `id` in `location.hosts` is resolved against `monitors` and emitted as one `converters` entry. The converter `type` and its `params` are written next to the converter; the verdict `type` and its `params` are written inside the nested `verdict` block:

```
converters:
- type: <monitor.converter.type>
  inputs: <resolved input names>
  <monitor.converter.params>
  verdict:
    type: <monitor.verdict.type>
    <monitor.verdict.params>
    exporters: <verdict exporters, see below>
```

7.4 Input binding

`ResolveInputNames` maps the monitor's `consumes` ids to the `name` values of the corresponding observations, producing the `inputs` list. `inputs` is a source-name filter applied before the converter, not a semantic description of the property. A multi-source monitor lists multiple names; the custom converter decides how to interpret the combined records. When a monitor consumes upstream verdicts (aggregation), its inputs are bound to the verdict source instead.

7.5 Link wiring

Links, and only links, produce the inter-location plumbing. There is no separate, mode-specific wiring step.

- **Ingress, records.** A link with `to` equal to the current location and `payload=records` produces the `verdict_runner.source` block from its `transport`. This is what makes the file a `verdict_runner` configuration.
- **Egress, records.** A link with `from` equal to the current location and `payload=records` attaches a `DataRecord` exporter. If the link `carries` all captured observations it is a global `exporters` entry; if it carries a subset, it is attached per source.
- **Egress, verdicts.** A link with `from` equal to the current location and `payload=verdicts` adds a verdict exporter to the `verdict.exporters` list of every hosted monitor it `carries`.
- **Ingress, verdicts.** The receiving location hosts an aggregation monitor; its ingress `records` link uses a verdict-aware source so that incoming verdicts are wrapped as records for the aggregation chain.

```
# egress records link -> exporter on the producing location
exporters:
  - type: <link.transport.type>          # mqtt | file
    <link.transport settings>

# ingress records link -> source on the consuming location
verdict_runner:
  source:
    type: <link.transport.type>          # mqtt | file | verdict-aware mqtt
    <link.transport settings>
```

A location with no links is fully self-contained: its captured observations and hosted monitors live in the same runtime configuration, which is exactly the traditional, centralised organisation.

8 Worked Mapping

The table summarises how the four organisations of Section 4 emerge from the plan, in terms of generated files and the rules that fire. The concrete paths, MQTT topics, and transport settings are filled from each link's `transport` object.

Organisation	Generated files	Plan / rules that fire
Traditional / centralised	one file	one location captures and hosts everything; no links.
Orchestrated	one file per capture location + one analysis file	capture locations: captured observations + egress records links; analysis location: ingress records link + all hosted monitors.
Choreographed	one file per robot + one aggregator file	robot locations: captured observations, hosted local monitors, egress verdicts links; aggregator: ingress verdict-aware source + hosted aggregation monitors.
Hybrid	one file per robot + one analysis file	robot locations: captured observations, hosted local monitors, <i>and</i> egress records links; analysis location: ingress records link + central monitors.
Offline replay	one recorder file + one replay file	recorder: captured observations + egress records link over file ; replay: ingress records link over file + hosted monitors.